CS/ECE 4750/7750 HW #4: Minimax and Alpha-Beta Pruning

Name: _____ Four-in-a-row on a 6 x 6 board

1. Implementation

Language: **Python 3** (standard library only). Run `python main.py` to reproduce every number in this report; `python -m unittest` runs the tests. All tables below are copied from that program's output (`results.txt`).

- **Board & moves.** A flat list of 36 cells. Successors are the empty cells horizontally, vertically or diagonally adjacent to any piece, listed in the normal order (smaller column first, then smaller row) or its reverse. The search places a mark, recurses, and removes it (make/unmake).
- **Terminal test** (checked before the depth cutoff): the last move made 4-in-a-row gives +1000 / -1000 for a win / loss of the root player; a full board gives 0.
- **Heuristic h(n)** with the assignment's weights, from the root player's point of view. A run is a maximal line of exactly 2 or 3 same marks (a 3 is not also counted as 2s); an end is open if the next cell along that line is on the board and empty. Two open ends = two-side-open, one = one-side-open.
- **Minimax** and **alpha-beta** (AIMA-style MAX/MIN with alpha, beta) share the same successor function, utilities, heuristic and depth cutoff (depth d = d-ply tree, root at ply 0). At the root a move replaces the current best only if strictly better, so ties go to the first move in the order.
- **Nodes generated** = child states created by applying a move (root not counted). Alpha-beta creates children one at a time, so children after a cutoff are never generated. **CPU time** = `time.process_time()` around the search only.

2. Task 2: Minimax vs. Alpha-Beta and Move Ordering (State A, X to move, depth 4)

Method	Selected Move	Minimax Value	Nodes Generated	CPU Time (ms)
Minimax - normal	(row 4, col 5)	-71	87,190	2524.3
Alpha-beta - normal	(row 4, col 5)	-71	15,746	412.8
Alpha-beta - reverse	(row 4, col 5)	-71	4,476	122.2

Principal variation found by the search: X(row 4, col 5) -> O(row 5, col 6) -> X(row 5, col 4) -> O(row 2, col 2) (leaf h = -71).

Q1. Identical: the selected move (row 4, col 5) and the minimax value (-71). Alpha-beta only skips branches that provably cannot change the decision: a branch is cut once it is known to be no better, for the player choosing above it, than an alternative already found. With the same move ordering and tie-breaking (keep the first move that reaches the best value), alpha-beta returns exactly the same move and value as minimax. **Different:** the number of nodes generated and the CPU time. Minimax generated the whole 4-ply tree (87,190 nodes); alpha-beta generated 15,746 (82% fewer) and ran about 6x faster. (The values alpha-beta computes for non-best root moves can also differ, because a pruned move only gets a bound, not its exact value.)

Q2. Alpha-beta returns the exact minimax value for any move order: pruning only skips subtrees that cannot affect the root, and the max/min over the tree does not depend on the order children are visited. The order only decides how early good bounds are found. The best root move, (row 4, col 5), is the last of 16 root moves in normal order but the first in reverse order. In reverse order alpha becomes -71 immediately, so each remaining root move is cut off as soon as O finds one reply worth at most -71. In normal order X first tries

weak column-1 and column-2 moves (mostly -1000), alpha rises slowly, and less is pruned: 15,746 vs. 4,476 nodes. The move is also unchanged because it is the unique best (next best: -81); with ties, another order could pick a different, equally valued move.

3. Task 3: Search Depth (State B, X to move, alpha-beta, normal ordering)

Depth	Selected Move	Minimax Value	Nodes Generated	CPU Time (ms)
2	(row 2, col 2)	-77	95	2.5
3	(row 2, col 2)	-1	962	25.1
4	(row 2, col 2)	-208	6,746	169.6

Evidence printed by the program: exact backed-up values of the best root moves at each depth, and the line both players follow inside the search after the top two moves.

```
depth 2: (row 2, col 2)=-77, (row 5, col 5)=-77, (row 5, col 2)=-277, (row 3, col 2)=-284
    X(row 2, col 2)  ->  O(row 5, col 5)   (leaf = -77)
    X(row 5, col 5)  ->  O(row 2, col 2)   (leaf = -77)
depth 3: (row 2, col 2)=-1, (row 5, col 5)=-9, (row 5, col 2)=-57, (row 3, col 2)=-64
    X(row 2, col 2)  ->  O(row 3, col 2)  ->  X(row 3, col 4)   (leaf = -1)
    X(row 5, col 5)  ->  O(row 3, col 4)  ->  X(row 3, col 2)   (leaf = -9)
depth 4: (row 2, col 2)=-208, (row 5, col 5)=-298, (row 3, col 1)=-1000, (row 4, col 1)=-1000
    X(row 2, col 2)  ->  O(row 3, col 4)  ->  X(row 3, col 2)  ->  O(row 2, col 4)   (leaf = -208)
    X(row 5, col 5)  ->  O(row 3, col 4)  ->  X(row 3, col 2)  ->  O(row 2, col 4)   (leaf = -298)
```

Q3. [(r,c) = (row, column)] The selected move does not change: X plays (row 2, col 2) at depths 2, 3 and 4. O's diagonal (3,3)-(4,4) is open at both ends, (2,2) and (5,5); if X does not block one end, O makes it a two-side-open 3 that X cannot stop. Depth 4 sees this: X(3,1) → O(2,2) → X(1,1) → O(5,5) is a loss, so every non-blocking move scores -1000. At depth 2 the two blocks tie at -77 (X(2,2) → O(5,5) vs. X(5,5) → O(2,2)) and the tie-breaking rule picks column 2. At depths 3 and 4, (2,2) is strictly better (-1 vs. -9; -208 vs. -298) because it also builds X's column 2: X(2,2) → O(3,4) → X(3,2) → O(2,4) gives X a two-side-open 3 (2,2)-(3,2)-(4,2) (+100) instead of only an open 2 (+10) after X(5,5).

Disclosure: the code and this report were prepared with the help of an AI coding assistant (Claude Code), as permitted by the assignment. All results were produced by the submitted program and checked by its unit tests (alpha-beta = minimax on random positions, hand-computed heuristic values).